HỒ SƠ BÀN GIAO — Brain, vòng tiếp theo

1. Trạng thái, kiểm lại bằng máy ngày 22/10/2026
   1.1. Tám lỗi lint, đã sửa ngày 22/10/2026
   1.2. Lịch sử git sạch, và vì sao bộ quét vẫn kêu
   1.3. Một mục trong ARCHITECTURE.md đã lỗi thời theo chiều có lợi
2. Bảy việc
   V1. Vá bảy chỗ tài liệu và config lệch khỏi mã
   V2. Nối `VLLMServerClient` vào đường phục vụ
   V3. `SCHEMA MAP` khớp ERD của Body
   V4. Bộ đo RAG
   V5. Phiên đo lại trên Colab
   V6. Hai việc chỉ người thật gỡ được
   V7. Mảng vận tải, nay ở lại repo này
3. Người tiếp quản cần được cấp những gì
4. Ngày đầu tiên
5. Bốn câu hỏi hỏi trước mọi commit
6. Cái gì không làm
7. Chín quyết định của chủ dự án, chốt ngày 22/10/2026
8. Danh sách ký nhận

HỒ SƠ BÀN GIAO — Brain, vòng tiếp theo

Lập ngày 22/10/2026. Mốc mã nguồn: nhánh `feat/workflow-format-and-deterministic-core`, commit `c81554f`.

Hồ sơ này bàn giao Brain cho người tiếp quản. Nó gồm bốn thứ: trạng thái đã kiểm lại bằng máy, bảy việc kèm cổng nghiệm thu, những gì người tiếp quản cần được cấp, và chín quyết định chủ dự án đã chốt ngày 22/10.

Mảng vận tải ở lại repo này. Chủ dự án chốt ngày 22/10/2026: không tách repo riêng. Nên lớp nền dùng chung giữ nguyên một bản, và BAN_GIAO_VAN_TAI.md §B.5 về việc mang gì theo khi tách **không còn áp dụng**. Danh sách việc của mảng đó tóm ở V7.

> **Đọc theo thứ tự.** Người tiếp quản chưa biết gì về dự án thì bắt đầu ở SO_TAY_NGUOI_MOI.md, chạy được hệ thống trên máy mình, rồi quay lại đây. Người đã quen repo thì đọc thẳng mục 2 và 3.

Bản in để gửi đi: `docs/ban-in/GIAO_VIEC_BRAIN.pdf`, 12 trang. Sửa file `.md` này rồi chạy `powershell -File docs/dung-ban-in.ps1` để dựng lại trong cùng PR.

---

1. Trạng thái, kiểm lại bằng máy ngày 22/10/2026

Mọi dòng dưới đây chạy lại được. Cột cuối nói con số đó **không** chứng minh điều gì.

Hạng mục	Lệnh kiểm	Kết quả	Không chứng minh được gì
Bộ test	`ENV=LOCAL pytest -q`	**1003 passed, 1 warning, 107s**	Cách hiểu nghiệp vụ có đúng không. Đáp án của 12 ca gieo lỗi do chính dự án đặt
Kiểu tĩnh	`mypy` theo danh sách CI	**Sạch, 5 file**	CI chỉ kiểm 5 file, không phải cả repo
Lint	`ruff check src tests offline_training`	**Sạch.** 8 lỗi đã sửa ngày 22/10, xem 1.1	CI chỉ lint một tập con nên nó vốn không bắt được 8 lỗi đó
Secret ở HEAD	grep 5 mẫu	**Sạch**	
Secret toàn lịch sử	`git log --all -S`	**Sạch.** Xem 1.2	

1.1. Tám lỗi lint, đã sửa ngày 22/10/2026

```
offline_training/ANSER_train_v3_colab.ipynb : cell 45 : F821  Undefined name `doc_health`
offline_training/eval_ke_toan.py            : 41      : I001  import chưa sắp xếp
src/core/inventory.py                       : 4 chỗ   : E741  tên biến mơ hồ `l`
src/core/period_diff.py                     : 1 chỗ   : E741  tên biến mơ hồ `l`
tests/test_auth_bao_phu.py                  : 19      : I001  import chưa sắp xếp
```

Bảy lỗi là chuyện hình thức, đã đổi `l` thành `dong` và `d`, và để `ruff --fix` sắp xếp import. Sau đó `pytest` vẫn 1003 passed.

Lỗi `F821` thì khác, và nó là một quả mìn hẹn giờ cho phiên đo ở V5. Ô 4.2 của notebook gọi `doc_health(timeout=2.0)` ở dòng 44 để kiểm xem còn uvicorn cũ nào đang giữ cổng 8000, trong khi hàm đó được định nghĩa mãi ở dòng 65 **của cùng ô**. Chạy tới đó là `NameError`, giữa buổi, sau khi đã nạp xong model. Đã chuyển định nghĩa lên trước lần gọi đầu tiên.

Việc còn lại cho người tiếp quản: mở rộng danh sách `ruff` trong `.github/workflows/ci.yml` ra cả `src`, `tests`, `offline_training`, để tám lỗi này không quay lại. Hiện CI chỉ lint chín file nên nó vốn không nhìn thấy chúng.

1.2. Lịch sử git sạch, và vì sao bộ quét vẫn kêu

Quét toàn bộ lịch sử theo đúng quy trình R2b cho ra: `postgres://` 7 commit, `sk-` 7 commit, `npg_` 3 commit, `AIza` 2 commit, `hf_` 1 commit.

Kiểm từng cái thì **không có credential thật nào**. Tất cả là fixture giả dùng để test chính bộ quét secret:

Chuỗi	Ở đâu	Là gì
`postgresql://admin:***@ep-cool-1.neon.tech/db`	`tests/test_training_pipeline_v3.py`	fixture, bắt buộc `scan_secrets` phải bắt được
`postgres://read_replica_user:***@replica_host`	cùng file	fixture, bắt buộc `scan_secrets` **không** được báo oan, vì mật khẩu là chữ `pass`
`npg_` kèm chuỗi giả	cùng file	fixture
`postgresql://admin:***@ep-x.neon.tech/db`	`tests/test_assistant_upgrades.py`	fixture

Mật khẩu trong bảng trên đã che. Chúng là chuỗi bịa, không phải credential thật, nhưng chép nguyên vào một file nữa chỉ làm bộ quét kêu thêm một chỗ mà không cho ai biết thêm điều gì. Cần đối chiếu chính xác thì mở hai file test nêu ở cột giữa. | mẫu `postgres://.*:.*@`, `npg_`, `sk-` | `AGENTS.md` dòng 60 | văn bản quy tắc |

Đợt dọn `01aba0b` ngày 27/07/2026 đã gỡ hết credential thật khỏi 12 file.

> **Việc của người tiếp quản:** trước khi push lịch sử lên một remote mới hoặc public, vẫn chạy quét. Nhưng biết trước năm dòng trên là fixture, để không mất một buổi điều tra lại thứ đã sạch. Và nếu có dòng nào **ngoài** danh sách đó, dừng lại ngay.

1.3. Một mục trong ARCHITECTURE.md đã lỗi thời theo chiều có lợi

ARCHITECTURE.md §13b.3 liệt kê "bốn lỗ hổng của `knowledge.py` phải vá trước khi nạp dữ liệu thật". Đọc mã ngày 22/10 thì **cả bốn đã vá**:

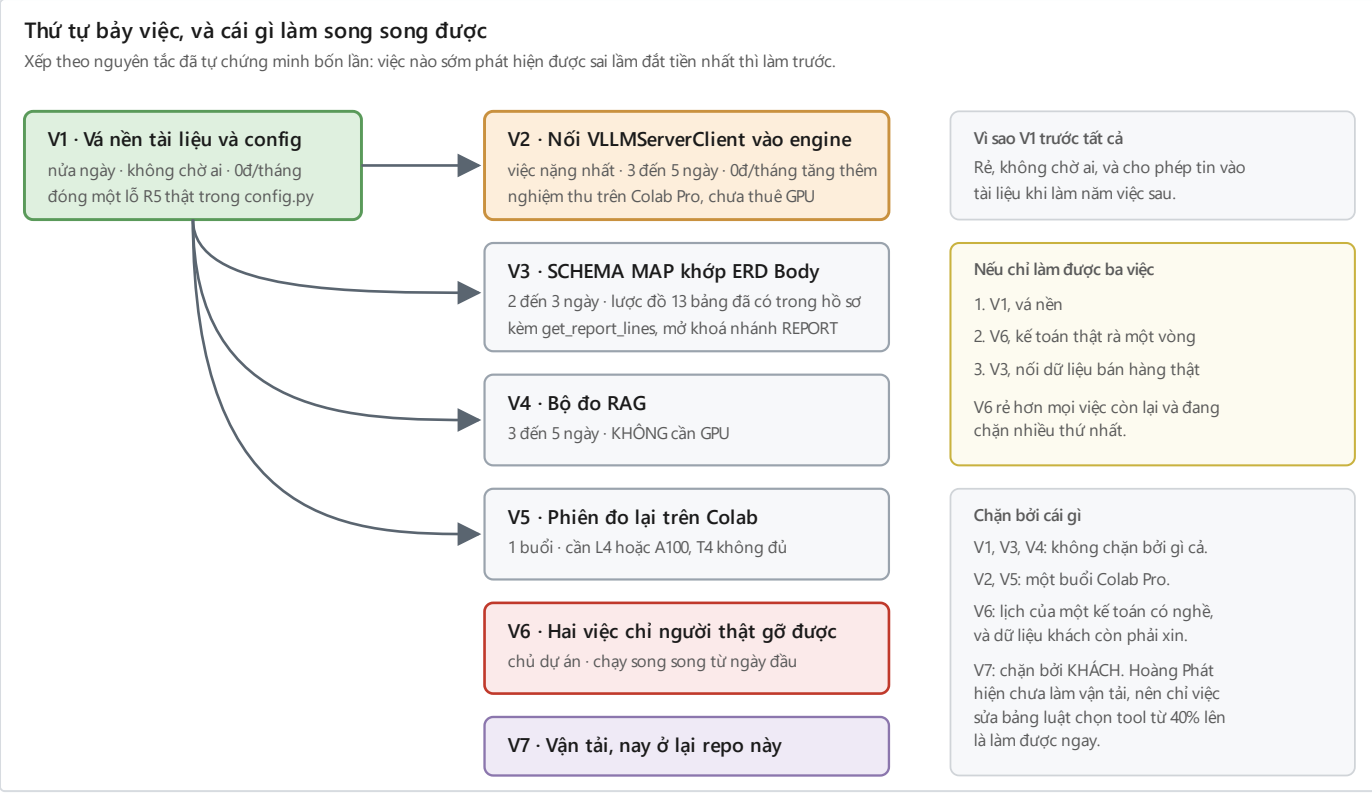
Lỗ hổng theo tài liệu	Trạng thái thật trong `src/core/knowledge.py`
Không trích dẫn được nguồn	`search()` trả `list[Passage]`, có `source`, `heading`, `score`
Không cô lập theo khách	`workspace_id` là tham số bắt buộc ở mọi lối vào, không có mặc định, thiếu thì `raise ValueError`
Không có hiệu lực theo thời gian	`Passage` có `effective_from` và `effective_to`; tài liệu hết hiệu lực bị **loại**, không phải xếp hạng thấp

Lỗ hổng theo tài liệu	Trạng thái thật trong `src/core/knowledge.py`
`ingest_folder()` nằm trong `__init__`	Đã gỡ. Dòng 301 ghi rõ lý do

Người tiếp quản đọc §13b.3 rồi đi vá lại là mất vài ngày cho việc đã xong. V4 vì vậy chỉ còn phần **đo**, không còn phần vá.

---

2. Bảy việc



V1. Vá bảy chỗ tài liệu và config lệch khỏi mã

Vì sao làm trước: rẻ, không chờ ai, và đóng một lỗ R5 thật. Làm xong thì năm việc sau đứng trên nền đáng tin.

#	Lệch gì	Ở đâu	Hậu quả
1	`TEXT_MODEL_ID` mặc định là đường dẫn Google Drive trỏ vào model v2 đã nghỉ hưu	`config.py:55`	Vi phạm R5, và trái §5.3 vốn chốt model nền là `Qwen/Qwen3-8B`. Ai triển khai theo mặc định sẽ nạp đúng bản mà phiên đo 15/08 kết luận "không dùng lại", bản bịa số 8 trên 27 ca ở narration
2	AGENTS.md và ARCHITECTURE.md mô tả Body là Flask, Redis + RQ, `workflow_engine.py`	AGENTS.md §1, ARCHITECTURE.md §6	Body thật là Next.js. Hai file nguồn sự thật đang mô tả một hệ không còn tồn tại

#	Lệch gì	Ở đâu	Hậu quả
3	§1 và §2.2 trỏ vào bốn module không tồn tại: `core/tools.py`, `agents/researcher.py`, `integrations.py`, `external_data.py`	ARCHITECTURE.md:34, :86, :96	`core/tools.py` được nêu làm ví dụ minh hoạ R1, ràng buộc số một, và file đó đã bị xoá
4	Mô tả "Project A", entrypoint `src/server.py`, cổng chất lượng chạy trên `core/tools.py`	`src/README.md`	Người mới làm theo README sẽ gãy ở bước chạy server và bước lint
5	Docstring "tối ưu cho Google Colab Pro, GPU L4"; log `"Booting COLAB engine — target GPU L4 22.5GB"`	`config.py:7`, `engine.py:188`	Trái §4.4 vốn đã loại Colab khỏi lộ trình production
6	Cảnh báo "config.py đang để 4096"	`deploy/.env.example`	Mặc định nay là 8192. Một cảnh báo sai lệch còn tệ hơn không có cảnh báo, vì lần sau không ai tin nó
7	§13b.3 liệt kê bốn lỗ hổng RAG như thể chưa vá	ARCHITECTURE.md §13b.3	Xem 1.3. Người tiếp quản đi vá lại thứ đã xong

Phần lint đã làm xong ngày 22/10, xem 1.1. Việc còn lại là mở rộng danh sách `ruff` trong `ci.yml` để chúng không quay lại.

Cổng nghiệm thu. Cả năm dòng dưới đều phải đúng:

```
grep -rn "content/drive" src/            # rỗng
ruff check src tests offline_training    # 0 lỗi
ENV=LOCAL pytest -q                      # 1003 passed
```

Mọi đường dẫn file trong ARCHITECTURE.md §1 và §2.2 tồn tại. Lệnh trong `src/README.md` chạy được như viết.

Công: nửa ngày. **Chi phí:** 0đ/tháng. **Chặn bởi:** không gì.

Cạm bẫy: mục 1 đừng thay bằng một HF id cứng khác. R5 đòi đọc từ env với mặc định an toàn, và mặc định an toàn ở đây là `Qwen/Qwen3-8B`, tải được từ Hub, khớp §5.1.

V2. Nối `VLLMServerClient` vào đường phục vụ

Đây là phát hiện mới ngày 22/10/2026, chưa có trong TIEN_TRINH_BRAIN.md §3.

Một lớp đã dựng xong, có 395 dòng test, được khai trong compose, và không caller nào gọi nó. Kiểm trong một phút:

```
grep -rn "VLLMServerClient" --include=*.py src/ tests/
  src/core/serving.py:241   class VLLMServerClient:
  src/core/serving.py:375   "VLLMServerClient",
  tests/test_serving.py     (6 chỗ)
```

`engine.py:9` import từ `serving` đúng một thứ, `guard_from_env`. Đường sinh text vẫn là:

```
# engine.py:195   API chạy lô NGOẠI TUYẾN, nạp model trong tiến trình
self.llm = LLM(model=self.config.text_model_id, ...)
# engine.py:268
async with self.text_guard.slot():
    return await loop.run_in_executor(None, _blocking_generate)
```

Nút thắt mà ARCHITECTURE §5.4 mô tả còn nguyên: continuous batching của vLLM vẫn bị vô hiệu hoá, và nhiều luồng vẫn gọi một object không thread-safe. Thứ đã làm xong là `ConcurrencyGuard`, nó chặn quá tải và trả 503 kèm `Retry-After`. Hữu ích, và nó chữa triệu chứng "treo" chứ không chữa nguyên nhân "tuần tự".

Ba hệ quả làm `deploy/docker-compose.yml` không chạy nổi nhánh `/chat`:

1. Compose đặt `ENV: SERVER` cho service `brain`, nhưng mã chỉ rẽ nhánh ở `== "LOCAL"` (`engine.py:181`). Mọi giá trị khác `LOCAL` đều đi vào nhánh nạp model lên CUDA trong tiến trình, ngược hẳn thiết kế "brain CPU only" vẽ ở đầu file compose.
2. Khối `environment:` của `brain` không đặt `TEXT_MODEL_ID`, chỉ có `TEXT_MODEL_DIR` cho service `vllm-text`. Nên `Config` lùi về mặc định ở V1 mục 1, tức đường dẫn Drive. Kết quả: `engine_error`, `/health` báo `degraded`, `POST /chat` trả 503.
3. `deploy/Dockerfile` ghi "KHÔNG có CUDA, image ~1GB" rồi `pip install -r requirements.txt`, mà file đó ghim `vllm>=0.10.2,<0.12` và `torch>=2.4.0`. Image sẽ kéo về vllm và wheel CUDA, phản lại chú thích của chính nó.

> **Phần tất định vẫn sống.** `/health` và `/tools/*` không cần model, nên lớp kế toán, thứ đang ra tiền, không bị chặn bởi việc này. Đó là lý do nó xếp sau V1.

Việc phải làm, theo thứ tự:

#	Việc
1	`engine.py` rẽ nhánh theo **backend** thay vì theo tên môi trường: có `VLLM_BASE_URL` thì dùng `VLLMServerClient`; không có thì nạp trong tiến trình, giữ đường cũ cho Colab; `ENV=LOCAL` thì mock. Ba đường, một chỗ quyết định
2	Hiểu `ENV=SERVER` tường minh, hoặc bỏ nó khỏi compose. Một giá trị không mã nào hiểu là một lời hứa suông
3	Đặt `TEXT_MODEL_ID` và `VLLM_MODEL` cho `brain` trong compose, khớp `--served-model-name` của `vllm-text`, hiện là `anser-v3`
4	Tách `requirements.txt` thành phần Brain chạy CPU và phần GPU hoặc train. Dockerfile chỉ cài phần đầu
5	Chỉ sau khi bốn bước trên xong mới nối `redis` cho `TASK_REGISTRY` và client HTTP cho vision

Cổng nghiệm thu, và đây là phần quan trọng nhất của việc này.

Viết một phép thử chứng minh đường HTTP **được thi hành**, không chỉ **tồn tại**. Phép thử hiện có không bắt được lỗ này:
`tests/test_deploy_config.py::test_moi_bien_compose_dat_deu_duoc_code_doc_hoac_khai_bao_la_chua_noi`
quét bằng regex, thấy `os.getenv("VLLM_BASE_URL")` nằm trong chính `serving.py`, rồi kết luận biến "đã được mã đọc". Đọc khác dùng.

Phép thử đúng dựng một server giả, gọi `generate_chat`, và **đòi server giả nhận được request**. Thấy lưu lượng mới là thi hành.

Nối xong thì `VLLM_BASE_URL` và `VLLM_MODEL` phải ra khỏi diện nghi vấn, còn `REDIS_URL` và `VLLM_VISION_*` vẫn nằm trong `CHUA_NOI` kèm lý do.

Thêm một cổng đo được: `inference_latency_p95` ở 4 request đồng thời, đo **trước** và **sau**. Ngưỡng ở ARCHITECTURE §13 là 5.000ms. Không đo trước thì không chứng minh được gì.

> **Nghiệm thu trên Colab Pro, chưa thuê GPU.** Quyết định của chủ dự án ngày 22/10: thuê máy thì lâu, trước mắt chạy trên Colab Pro. Hệ quả cụ thể cho việc này: dựng `vllm serve` trong một ô notebook, Brain gọi vào `127.0.0.1:8001`, và đo p95 ở đó. Đủ để chứng minh đường HTTP thi hành và continuous batching có chạy.
>
> Thứ Colab **không** chứng minh được: hành vi khi để chạy dài ngày, và chính bản compose. Điều khoản Colab Paid cấm phục vụ web service, nên `docker compose up` trên máy thuê vẫn là việc chưa ai chạy thử. Ghi rõ điều đó lúc báo cáo kết quả, đừng để người đọc tưởng compose đã được kiểm.

Công: 3 đến 5 ngày, cộng một buổi Colab để nghiệm thu. **Chi phí:** 0đ/tháng tăng thêm. Đổi lại: throughput tăng theo số request đồng thời, restart Brain không nạp lại 6GB weights, và vLLM sập thì `/tools/*` vẫn sống. **Chặn bởi:** không còn gì, Colab Pro đã có.

V3. `SCHEMA MAP` khớp ERD của Body

Đã gỡ chặn ngày 22/10/2026, và lược đồ thật đã đọc được. Nguồn: nhánh `docs/onboarding-playbook` của `Dgsonn/ANSER-v2`, file `frontend/src/server/db/schema.ts`, 13 bảng.

> Chủ dự án nói rõ: ERD **đã chốt nhưng chưa hoàn thiện, còn một giai đoạn thử nghiệm**. Nên sửa theo hướng rẻ tiền khi nó đổi tiếp: giữ mọi định danh trong đúng khối `SCHEMA MAP`, đừng rải tên cột vào SQL, và thêm một phép thử hợp đồng kêu to khi Body đổi.

Ánh xạ, đọc thẳng từ `schema.ts`

Brain đang khai	Body thật	Ghi chú
`PRODUCTS_TABLE = "products"`	`products`	đúng
`P_ID`, `P_NAME`, `P_PRICE`	`id`, `name`, `price`	đúng. `price` là `integer`
`P_STOCK = "stock_quantity"`	**`stock`**	sai. Kiểu `numeric(14,3)`, không phải số nguyên
`P_WORKSPACE = "workspace_id"`	**`warehouse_id`**	sai. Kiểu `uuid`, NOT NULL, FK tới `warehouses.id`
`SALES_TABLE = "sales"`	**`sales_invoices`**	sai. Dòng hàng nằm ở `sales_invoice_items`
`S_AMOUNT = "amount"`	**`total`**	sai. Kiểu `integer`
`S_DATE = "created_at"`	`created_at`	đúng
`S_WORKSPACE = "workspace_id"`	**`warehouse_id`**	sai, và **nullable**, xem cạm bẫy dưới

Body **không có khái niệm** **`workspace`** ở đâu cả. Cột phân phạm vi duy nhất là `warehouse_id`, có mặt trên `products`, `inventory_transactions`, `sales_invoices`, `employees`, `automation_rules`.

> `Config.kb_workspace_id` là chuyện khác và **phải giữ nguyên**. Đó là phạm vi kho tri thức của riêng Brain, cố ý tách khỏi kho hàng, vì hợp đồng và bảng giá cước là của cả công ty. Đừng gộp hai thứ chỉ vì tên gần giống nhau.

Ba cạm bẫy trong chính lược đồ đó

1. **`sales_invoices.warehouse_id` nullable.** Câu `WHERE warehouse_id = :ws` sẽ **âm thầm bỏ** mọi hoá đơn chưa gán kho. Không lỗi, không cảnh báo, chỉ là doanh thu thiếu đi một phần. Đúng loại lỗi "đúng số, sai câu hỏi" mà dự án đã dính một lần với kỳ báo cáo. Quyết định có chủ đích: hoặc `warehouse_id = :ws OR warehouse_id IS NULL`, hoặc đếm riêng số hoá đơn không có kho rồi báo ra.
2. **`products.stock` là `numeric(14,3)`,** không phải số nguyên. Ép kiểu sai làm tròn mất phần lẻ, mà dầu nhớt bán theo lít có phần lẻ thật.
3. **`warehouses.allows_zero_value`.** Body có cột này, Brain có `allow_zero_value` trong `audit_inventory`. Cùng một khái niệm khai ở hai repo, đúng thứ R4 cấm. Chốt một nguồn: Brain đọc cờ từ Body thay vì nhận qua tham số, hoặc ghi rõ vào tài liệu rằng tham số của Brain phải lấy từ cột đó.

Và một thứ đáng tiền hơn cả việc sửa bốn định danh

`sales_invoice_items` có `unit_cost`, `quantity`, `line_total`, `product_name`. `sales_invoices` có `total`, `due_date`, `customer_id`. `invoice_payments` có `amount` và `paid_at`.

Nghĩa là hai thứ Brain đang nói "chưa nối được" **nay nối được**:

Thứ	Hiện trạng	Có gì trong Body
`SaasAPI.get_report_lines`	**chưa có hàm.** `chat.py` kiểm `getattr(saas, "get_report_lines", None)` rồi trả "Tôi chưa nối được với dữ liệu bán hàng"	`sales_invoice_items` cho đủ dòng bán kèm **giá vốn từng dòng**, thứ `reporting.py` phải tự thú là chỉ phủ 62%
Tuổi nợ 30/60/90	ROADMAP ghi là chặn, chờ sổ chi tiết công nợ của MISA	`sales_invoices.due_date` cộng `invoice_payments` cho ra tuổi nợ **cho dữ liệu nhập trong ANSER**

> **Đừng vội mừng về tuổi nợ.** Số thật của Hoàng Phát đang nằm trong MISA, không nằm trong Body. Đường này chỉ phục vụ dữ liệu họ nhập vào ANSER. Nó không thay việc xin sổ chi tiết công nợ ở V6, nó chỉ làm tính năng sẵn sàng từ trước.

Việc phải làm

1. Sửa bốn định danh trong `SCHEMA MAP`, không rải ra chỗ khác.
2. Quyết cách xử `warehouse_id IS NULL` và ghi lý do ngay tại chỗ.
3. Viết `SaasAPI.get_report_lines` đọc `sales_invoice_items`, trả đúng hình dạng `rp.SaleLine` và `rp.ExpenseLine` mà `_build_report_context` đang chờ.
4. Cập nhật cả ba nơi theo R4: `prompts.py::DB_SCHEMA`, `offline_training/db_schema.txt`, SQL trong `saas_api.py`. Việc này đóng luôn bug ARCHITECTURE §11.5 mục 1.
5. Thêm phép thử hợp đồng vào `tests/test_body_contract.py`: đọc `schema.ts` (hoặc một bản chụp đã commit) rồi đối chiếu từng định danh trong `SCHEMA MAP`. ERD còn đổi, nên thứ cần là một phép thử **kêu to lúc nó đổi**, không phải một lần sửa đúng.

Cổng nghiệm thu. Nhánh DATA_INTERNAL trả số thật cho cả ba kỳ `hôm nay`, `tuần này`, `tháng này`, và **vẫn nói thẳng "chưa lấy được"** cho "tháng trước", đúng kịch bản 5.4 của CHAY_THU_NOI_BO.md. Vế sau quan trọng ngang vế trước. Thêm: câu "quý này lãi hay lỗ" đi hết vòng agentic và ra số thật thay vì câu "chưa nối được dữ liệu bán hàng".

Công: 2 đến 3 ngày, tăng so với ước tính cũ vì có thêm `get_report_lines`. **Chi phí:** 0đ/tháng. **Chặn bởi:** không còn gì. Lược đồ đã có trong hồ sơ này.

V4. Bộ đo RAG

`src/core/knowledge.py` có 571 dòng và 70 test đơn vị, và không một chỉ số recall, MRR hay faithfulness nào. Bốn lỗ hổng chức năng đã vá hết, xem 1.3, nên việc còn lại thuần là đo.

Phương án chi tiết ở BAN_GIAO_VAN_TAI.md §E.3, theo đúng nó trừ một chỗ:

> **E.3 bước 2 vi phạm R2 nếu áp nguyên văn.** Nó đề nghị "với mỗi đoạn, sinh 2 đến 3 câu hỏi, dùng DeepSeek", và danh sách tài liệu nguồn ngay phía trên là "hợp đồng, bảng giá cước, chính sách công nợ". Đó là dữ liệu khách hàng, và bảng giá cước là một trong hai thứ nhạy cảm nhất hệ thống.
>
> **Cách làm đúng:** DeepSeek chỉ cho corpus công khai, tức nghị định và văn bản pháp luật, nơi `legal_miner.py` đã có khung. Tài liệu của khách thì gõ tay câu hỏi, hoặc sinh bằng model local. Nếu buộc phải chọn một, 30 câu gõ tay trên tài liệu khách giá trị hơn 300 câu trên nghị định, vì đó mới là thứ khách hỏi.

Ba việc cải tiến giữ nguyên thứ tự của E.3: chỉnh cách chia đoạn, vì bge-m3 nhận tới 8192 token và chia theo 128 token của model cũ là phí; bật tắt ViRanker rồi so; tìm lai từ khoá cộng vector. Việc thứ ba đáng chú ý với ANSER hơn hầu hết hệ thống khác, vì khách tra bằng mã hàng như `VT00039` và biển số xe, hai thứ nhúng vector xử lý tệ.

`grounding.py` đã là một dạng faithfulness thu hẹp về con số, chặt hơn bản tổng quát ở đúng chỗ quan trọng nhất với sản phẩm tài chính. Việc cần làm là mở rộng nó thành chỉ số báo cáo được.

Cổng nghiệm thu. Recall@20 từ 0,8 trở lên, MRR có báo cáo, faithfulness có báo cáo. Người rà lại 20% mẫu, và câu hỏi sinh máy mà tự trả lời được không cần truy hồi thì phải loại.

Công: 3 đến 5 ngày. **Chi phí:** 0đ/tháng, embedder và reranker chạy CPU. Dùng DeepSeek cho corpus pháp luật thì vài chục nghìn đồng một lần, không phải chi phí thường xuyên. **Chặn bởi:** không gì, trừ việc cần tài liệu thật của khách để bộ đo có ý nghĩa.

V5. Phiên đo lại trên Colab

Đóng việc treo số 1 và số 2 của TIEN_TRINH_BRAIN.md §3. Runbook đầy đủ: PHIEN_DO_COLAB.md. Khoảng một giờ khi đo lại, vì dữ liệu sinh phiên 15/08 còn trên Drive tại `ANSER_AI_Logistics/generated`. Chạy lại ô 13 đến 15 là gọi DeepSeek lần nữa để nhận đúng thứ đang có: mất tiền, mất 35 phút, đổi lại không gì cả.

Hai thứ cần nhìn sau phiên này:

1. `render_tools()` đã thôi quảng cáo `sales`, `expenses`, `lines*`, những trường hệ thống tự bơm mà lược đồ đã gạch tên. Sửa rồi, chưa có số chứng minh nó kéo agentic lên. Agentic 15,8% phần lớn là lỗi lược đồ.

2. `nha.rang_buoc` của `providers.py` ghi lại chế độ ràng buộc **dự định**; `smoke_test_guided()` kiểm chế độ đó có **thi hành** thật không. Phiên 15/08 sửa `additionalProperties: False` mà kết quả không đổi một đơn vị nào, vì backend giải mã bỏ qua ràng buộc đó. Cả một vòng đo mất trắng.

Trước khi mở notebook, sửa lỗi `F821` ở ô 45 (xem 1.1), nếu không nó nổ giữa buổi sau khi đã đốt GPU.

Cổng nghiệm thu. Ba dòng cảnh báo khung đo được đọc **trước** điểm số: `⚠ đầu ra KHÔNG đọc được thành JSON`, `⚠ CẮT CỤT`, `[chốt chặn]`. Tỷ lệ JSON hỏng từ 30% trở lên gần như luôn là mã chứ không phải model. Kết quả so theo cặp bằng `compare_runs.py`, không so hai điểm trung bình.

Công: một buổi. **Chi phí:** một buổi Colab Pro, không phải chi phí thường xuyên. **Chặn bởi:** GPU L4 hoặc A100; T4 16GB không đủ cho Qwen3-8B AWQ.

Nhắc: notebook đã có ở `offline_training/ANSER_train_v3_colab.ipynb`. Chỉ thị của chủ dự án là không tự tạo notebook mới.

V6. Hai việc chỉ người thật gỡ được

Đưa vào hồ sơ để chúng không rơi khỏi danh sách chỉ vì không ai viết được dòng mã nào cho chúng. Chạy song song từ ngày đầu.

Việc	Ai gỡ	Vì sao không hoãn được
Một kế toán có nghề rà HOANG_PHAT_SOAT_SO_1108.md một vòng	chủ dự án tìm người	Bộ đo trả lời được "mã có làm đúng thứ ta định không", và không trả lời được "cách hiểu nghiệp vụ của ta có đúng không". Ba chỗ đang tự khẳng định: bảng tra thuế 8% và 10%, định nghĩa "sửa hồi tố", ngưỡng tập trung 20% và 50%. Sai bảng thuế thì khách xuất hoá đơn sai thuế suất, dẫn tới truy thu và phạt
Xin nốt dữ liệu còn thiếu theo HOANG_PHAT_DU_LIEU_CAN_XIN.md	chủ dự án và khách	Sổ chi tiết công nợ mở khoá tuổi nợ 30/60/90, thứ khách cần nhất trong bức tranh phải thu 3,96 tỷ. Không có ngày hoá đơn thì số dư hiện có không nói được ai quá hạn

Việc đầu mất một buổi và rẻ hơn mọi việc còn lại trong hồ sơ này.

V7. Mảng vận tải, nay ở lại repo này

Chủ dự án chốt ngày 22/10: **không tách repo.** Hai hệ quả tức thì.

Hệ quả tốt: lớp nền giữ một bản. `findings.py`, `tool_planner.py`, `agentic.py`, `grounding.py`, và `src/api/` kèm `auth_guard` không phải chép tay sang repo thứ hai, nên không có chuyện hai bản trôi xa nhau. Bản vá xác thực ngày 15/08 cũng không cần mang đi đâu.

Hệ quả phải nhớ: mảng vận tải có **82 test** và **chưa một con số nào được xác nhận bằng dữ liệu thật**. Mã đã lắp xong, máy chưa nổ. Đừng để con số test làm mình tin nhầm.

Bàn giao chi tiết ở BAN_GIAO_VAN_TAI.md, đọc Phần C và D. Tóm tắt thứ tự:

#	Việc	Vì sao xếp ở đó
1	**Xin 15 đến 20 báo giá đã chốt** rồi chạy `calibration.replay_pricing()`	Cánh cổng chưa bao giờ mở. MAPE dưới 10% là công thức dùng được, trên 25% là mô hình sai bản chất chứ không phải sai tham số. Không có số này thì mọi thứ khác xây trên cát
2	**Xin 10 ca chọn nhà xe trong quá khứ**, chạy `CarrierChoiceCase`	Sáu trọng số hiện do đội gốc tự đặt. Trùng dưới 6 trên 10 thì hỏi khách vì sao, rất có thể có một tiêu chí thứ bảy không nằm trong mã
3	**Sửa bảng luật chọn tool**, đang 40% trên bộ eval vận tải	Mã tất định trong `tool_planner.py`, sai thì sửa regex chứ không train lại. 40% nghĩa là cứ 5 câu báo giá thì 3 câu đi nhầm đường
4	**Nối giá dầu tự động**	Giá đổi chiều thứ Năm hằng tuần theo NĐ 80/2023. Báo giá phát hành thứ Hai có thể sai vào 15 giờ thứ Năm, nên báo giá phải có hạn hiệu lực
5	**Dựng màn hình báo giá** bên Body	Hiện chưa có gì, toàn bộ mã vận tải chỉ gọi được qua API. Việc này thuộc Body nên chỉ được **đề xuất bằng văn bản**
6	**Sinh giấy vận tải điện tử** theo NĐ 158/2024	Dữ liệu gần như đã có sau khi chốt báo giá. Bản bàn giao đánh giá đây là tính năng bán được nhất, và không đối thủ nào trong bảng A.4 làm

Một khác biệt về model phải nhớ. Kết luận "dùng model gốc" chỉ đúng cho nhánh kế toán. Ở nhánh vận tải thì ngược lại: extraction đo được model gốc **0,0%**, bản fine-tune **65,3%**, sửa 64 câu và làm hỏng 0. Hai kết luận ngược nhau cho hai mảng, và cả hai đều đúng. Bản AWQ nằm trên Drive tại `ANSER_AI_Logistics/anser-v3-awq`.

Giới hạn của con số 65,3%: nó đo trên dữ liệu **sinh ra**, không phải tin nhắn thật của khách. Người thật viết tắt, sai chính tả, nhắn nhiều tin rời rạc. Con số trên tin nhắn thật sẽ thấp hơn, và khoảng cách đó chính là việc phải làm.

Chặn bởi: việc 1 và 2 chặn bởi khách, và khách hiện **chưa làm vận tải**. Nên V7 chỉ khởi động được khi có một khách vận tải thật, hoặc khi Hoàng Phát mở mảng đó. Việc 3 làm được ngay, không chờ ai.

---

3. Người tiếp quản cần được cấp những gì

Danh sách này để chủ dự án tích từng dòng lúc bàn giao.

Cần	Dùng cho	Ghi chú
Quyền ghi `PCBoiz/ANSER_AI`	mọi việc	
Quyền đọc `Dgsonn/ANSER-v2`, nhánh `docs/onboarding-playbook`	V3, đọc `frontend/src/server/db/schema.ts`	Chỉ đọc là đủ. Người tiếp quản không được sửa Body
Tài khoản Colab Pro, có L4	**V2 và V5**	T4 16GB không đủ cho Qwen3-8B AWQ. Đây là nơi nghiệm thu V2, vì chưa thuê GPU
Quyền đọc Drive `ANSER_AI_Logistics/`	V5	Chứa `anser-v3-awq` và thư mục `generated` của phiên 15/08

Cần	Dùng cho	Ghi chú
`DEEPSEEK_API_KEY`	V4 corpus pháp luật, V5 nếu sinh dữ liệu mới	Dán thẳng vào Colab Secrets, **không** đi qua chat và không nằm trên máy làm việc
`DATABASE_URL` nhánh dev của Neon	V3	**Nhánh dev, không phải production**
Một kênh liên lạc với đội Body	V3	Để hỏi khi `schema.ts` không nói rõ
Bốn file MISA đã ẩn danh	chạy bộ đo kế toán	Bản gốc không vào repo

Chưa cần ở vòng này: tài khoản thuê GPU, token Cloudflare Tunnel, quyền ghi n8n production.

> **Theo AGENTS R2b mục 3**, đưa bất kỳ giá trị bảo mật nào vào env hoặc đổi cách quản lý bí mật đều phải hỏi chủ dự án trước. Đây là chỉ thị trực tiếp, không phải khuyến nghị.

---

4. Ngày đầu tiên

Một buổi, và kết thúc buổi là biết mình đã tiếp quản được hay chưa.

1. Đọc SO_TAY_NGUOI_MOI.md mục 1 và 2. Dựng môi trường, `ENV=LOCAL pytest -q` ra 1003 passed.
2. Bật uvicorn, `GET /health` trả `auth_enabled: false`. Đặt `API_AUTH_TOKEN` rồi xác nhận nó đổi thành `true`, và `GET /tools` không kèm token trả 401.
3. Sinh `ton_kho_mau.xlsx`, nạp qua `/tools/inventory-import`, đối chiếu 8 phát hiện trên 7 mã. Mở file Excel và kiểm tay một phát hiện bằng `evidence` của nó.
4. Đọc AGENTS.md phần R1 đến R5, và mục 9 của sổ tay.
5. Chạy ba lệnh kiểm ở mục 1 của hồ sơ này và xác nhận bạn ra cùng số.

Không ra được 1003 passed trong vòng một giờ thì đó là lỗi của tài liệu. Ghi lại ba thứ, câu lệnh đã gõ, thứ nhận được, thứ mong đợi, rồi sửa mục 2.9 của sổ tay.

---

5. Bốn câu hỏi hỏi trước mọi commit

Lấy từ AGENTS.md §5.2. Mỗi câu đứng sau một sự cố đã xảy ra:

1. Nếu model trả về rác, có bản ghi sổ sách nào sai theo không? (R1)
2. Payload này có chứa dữ liệu khách hàng, và nó đi tới đâu? (R2)
3. Việc này làm tăng chi phí GPU bao nhiêu VNĐ mỗi tháng? (R3)
4. Định nghĩa này đã tồn tại ở chỗ khác chưa? (R4)

Và hai nguyên tắc đọc số:

`None` là chưa biết, `0` là biết và bằng không. Gộp hai thứ này tạo ra báo cáo sai mà đọc rất xuôi. Báo cáo lãi lỗ sẽ coi mọi mặt hàng chưa nhập giá vốn là lãi 100%.

Đừng tin một lớp phòng thủ chỉ vì đã viết nó. V2 là lần thứ tư dự án dính đúng lỗi đó. Ba lần trước: `API_TOKEN` so với `API_AUTH_TOKEN`, `arguments_schema()` lọc trường nhưng không đóng object, và trần token benchmark rộng hơn production. Không test đơn vị nào bắt được cả bốn, và cả bốn lộ ra khi chạy thật qua HTTP hoặc khi đọc tay đầu ra.

---

6. Cái gì không làm

Không làm	Vì sao
Hiệu chỉnh công thức báo giá cước	Thuộc bản bàn giao vận tải
Train lại model cho nhánh kế toán	Đã đo: narration hỏng 11 sửa 0, p = 0,0010, trong đó 8 ca bịa số. Dùng model gốc
Text-to-SQL cho báo cáo	Số tài chính sai mà nghe hợp lý là loại lỗi tệ nhất
Đưa giá dầu, số dư công nợ, tồn kho vào RAG	Đổi hằng ngày; embedding cũ nằm lại và vẫn bị truy hồi ra
Fine-tune VLM	Chưa có một hoá đơn thật nào để đo
Chạy `src/evaluate_system.py` trên dữ liệu khách	Nó gọi DeepSeek làm judge, xem `evaluate_system.py:32`. Chỉ chạy trên dữ liệu tổng hợp hoặc mẫu. Ưu tiên thay bằng judge local
Đưa file MISA gốc vào repo	Chứa tên khách, mã số thuế, số nợ. Dùng bản ẩn danh, đã kiểm là cho kết quả y hệt
Gộp nhánh làm việc vào `main`	`main` là bản trước khi có lõi tất định
Sửa repo Body	AGENTS.md §1: chỉ được đề xuất bằng văn bản rồi dừng lại chờ người quyết định
Nâng `--workers` của uvicorn	`TASK_REGISTRY` còn là dict trong RAM
Dùng Colab phục vụ traffic thật	Điều khoản Colab Paid cấm web service

---

7. Chín quyết định của chủ dự án, chốt ngày 22/10/2026

Đây là những thứ không nằm trong mã và người tiếp quản không tự suy ra được.

#	Câu hỏi	Quyết định	Đổi gì trong hồ sơ này
1	Nhánh nào của Body là ERD đã chốt	Nhánh `docs/onboarding-playbook`. **Đã chốt nhưng chưa hoàn thiện, còn giai đoạn thử nghiệm**	V3 gỡ chặn hẳn, lược đồ 13 bảng đã đọc và chép vào V3. Vì còn đổi tiếp, V3 đòi thêm một phép thử hợp đồng
2	Đã tìm được kế toán rà sổ chưa	**Chưa**	V6 vẫn mở, và nó là việc rẻ nhất đang chặn nhiều thứ nhất. Nên khởi động trong tuần đầu vì phụ thuộc lịch người ngoài
3	Khi nào thuê GPU	**Chưa thuê.** Thuê thì lâu, trước mắt dùng Colab Pro	V2 nghiệm thu trên Colab. Bậc T0 và bản compose lùi lại, và phải nói rõ là compose vẫn chưa ai chạy thử

#	Câu hỏi	Quyết định	Đổi gì trong hồ sơ này
4	Mảng vận tải có ở lại repo này không	**Có**	Không tách repo. Lớp nền giữ một bản, §B.5 của bản bàn giao vận tải hết hiệu lực. Thêm V7
5	Còn khách nào ngoài Hoàng Phát	**Chỉ một**	Chưa cần cô lập dữ liệu đa khách. Thứ tự sáu việc giữ nguyên
6	Người tiếp quản là ai	**Một người mới vào, và một người sắp nghỉ một thời gian**	Tài liệu phải tự đứng được, xem ghi chú dưới bảng
7	Có commit bản PDF vào repo không	**Có**	`docs/ban-in/*.pdf` nằm trong repo. Sửa `.md` thì chạy lại `docs/dung-ban-in.ps1` trong cùng PR
8	Commit luôn hay chờ duyệt	**Commit luôn**	
9	Sửa 8 lỗi lint luôn không	**Có**	Đã sửa, xem 1.1. `pytest` vẫn 1003 passed

> **Điều quan trọng nhất rút ra từ mục 6.** Đội sắp gồm một người chưa biết gì về dự án, và người biết nhiều nhất thì sắp vắng. Nghĩa là **không hỏi được ai** sẽ là trạng thái bình thường, không phải ngoại lệ.
>
> Hệ quả cho cách làm việc: mỗi lần tìm ra một thứ không có trong tài liệu, viết nó vào tài liệu ngay trong cùng PR, kể cả khi nó chỉ là một dòng. Và khi mã với tài liệu mâu thuẫn, tin mã rồi sửa tài liệu, đừng để người sau tự đoán.
>
> SO_TAY_NGUOI_MOI.md viết cho đúng tình huống này: người đọc chưa biết gì, không có ai ngồi cạnh. Mục 2.9 của nó là bảng gỡ lỗi, mục 10 là bảng những chỗ tài liệu đang lệch khỏi mã.

---

8. Danh sách ký nhận

Một việc coi là xong khi đủ cả bảy, theo AGENTS.md §7:

- ☐ Mã chạy ở `ENV=LOCAL` không cần GPU
- ☐ Không vi phạm R1 đến R5
- ☐ Có test cho phần logic tất định; `pytest` xanh, hoặc nêu rõ cái nào đỏ và vì sao
- ☐ Tài liệu bị ảnh hưởng đã cập nhật trong cùng PR
- ☐ Nêu được chỉ số cải thiện và cách đo
- ☐ Nêu được tác động chi phí bằng VNĐ mỗi tháng
- ☐ `[PASSED BY MENTOR]`

Bàn giao coi là xong khi người tiếp quản làm được mục 4 mà không cần hỏi ai, và chạy ba lệnh kiểm ở mục 1 ra cùng con số.

Hồ sơ này viết từ việc đọc mã, không từ việc đọc tài liệu cũ, vì tài liệu cũ đã sai ở bảy chỗ nêu ở V1 và một chỗ nặng hơn nêu ở V2. Khi mã và tài liệu mâu thuẫn, tin mã, rồi sửa tài liệu trong cùng PR.